

DMRC Intern Referral Portal — Handover Brief

Prepared for: DMRC IT — deployment and intranet integration
Prepared by: Shivam Sinha
Status: Pilot. Not replacing any system currently in production.
Revised: 25 August 2026 — updated with the answers received to date.

1. What this is

A web application supporting the intern referral and onboarding process end to end:
referral → verification → offer letter → joining → clearance → completion certificate → dispatch

Part	Users	Access
Phase 1 — Referral portal	Any DMRC employee	Open to all identified staff
Phase 2 — HR dashboard	HR staff only	Three roles: HR-OPS, HR-APP, SYS-ADMIN

Offer letters and completion certificates are generated as PDF and Word files and stamped with the issuing officer's stored signature image. Every action is written to an audit ledger.

2. Technical summary

Component	Detail
Backend	Python / Django 4.2.30 with Django REST Framework
Database	MySQL 8.0.16 or newer, over an encrypted connection
Frontend	Static HTML, CSS and JavaScript (Alpine.js, Bootstrap 5)
Build step	None. No Node.js, no npm, no compilation.
External dependencies	None at runtime. All frontend libraries are held locally at pinned versions; neither page contacts the internet.
Application server	To be agreed — Gunicorn or uWSGI behind your web server
Deployment shape	Pages and API on separate servers, per your confirmation
File storage	Server filesystem, outside the web root (\$7)
API surface	28 endpoints, every one role-protected (\$5)

3. The integration

This portal authenticates nobody. No login screen, no stored passwords. Identity comes from DMRC's existing employee login; the portal only decides what an already-identified employee may do here.

DMRC employee login	→ WHO the person is	(your system)
This portal	→ WHAT they may do here	(this system)

Your stated approach: an encrypted token accompanying the URL, carrying the user's identity, with employee data queried directly on the application server. That fits the design — the entire integration is one method, `get_employee_code()`, in `portal/identity/intranet.py`. It returns the employee code of the signed-in user, or nothing for an unauthenticated request, in which case the portal replies 401.

`portal/identity/base.py` documents the contract in full. The token approach means implementing decrypt-and-extract in that one method; nothing else in the codebase changes.

Once a code is resolved, two tables decide everything:

- `employees` — any active employee found here may use the Phase 1 referral portal.
- `users` — dashboard accounts only. An employee with no row here cannot reach the HR dashboard at all. This is the default for all staff and is intentional. Roles are granted deliberately by a SYS-ADMIN, never inherited from the intranet.

4. Questions

Answered — recorded here so we are working from the same facts

Question	Your answer	What it means for me
Employee code format	Numeric, 1-5 digits (9, 24, 416, 3201, 14086)	Nothing to change — the column accepts it. See the note below.
Same server for pages and API?	No — separate servers	Frontend calls must be converted to a configurable API address (\$8, gap 1)
Does MySQL require encryption?	Yes	<code>DB_SSL=True</code> — I still need the CA certificate path (Q3)
Employee directory	A DMRC database of current employee records, refreshed nightly; direct connection available	Removes the need for a manual import. See Q7.
First SYS-ADMIN	Person identified; to be created at go-live	Note the sequencing point in \$6

One operational note on numeric codes. A one- or two-digit code is easy to mistype, and a typo lands on a *different real employee* rather than on nothing. The IAM screen mitigates this — it looks the code up and shows the name before granting anything — so whoever administers accounts should read the name back before confirming.

Outstanding

#	Question	Why it blocks me
1	What encryption scheme is the token, and how do I get the key or certificate?	Cannot write the integration without it.
2	What does the token carry? I need employee number, name, designation and department. All four, or just the number with the rest queried? Official email optional — I'll store it if available.	If the token carries all four, the employee record can be created on first sign-in and no separate sync is needed.
3	Where is the CA certificate file on the database server? If it is self-signed with no CA file, I will encrypt without certificate verification — but I would rather you confirmed that is acceptable.	The application cannot connect without this.
4	Does the token expire? A token in a URL appears in browser history and server logs. I would want a short validity window that I verify server-side.	Without expiry, anyone who obtains that URL becomes that employee indefinitely.
5	After the first page load, does every API call carry the same token, or should I exchange it once for a session? Happy to do either.	Determines roughly sixty lines of frontend work.
6	"Application server" — the one serving the pages, or the one running the API? They are separate, so the token has to reach the API side.	Determines where decryption happens.
7	If employee data is to be queried: which table, and which columns hold the four fields? Five sample rows would be ideal. Also — how are departed employees represented, and what time does the nightly refresh finish?	Determines the sync job.
8	What hostname will the portal be reached at, and what address will the API be on?	Needed for <code>ALLOWED_HOSTS</code> , the CORS list, and the frontend's API address.
9	Which Linux distribution, and what does <code>python3 --version</code> print on the target server?	Scopes the framework upgrade in \$5.
10	Which MySQL version?	One validation rule is enforced from 8.0.16 onward and silently ignored before it.
11	What browsers do HR staff use, and which versions?	The interface requires a modern browser.
12	Mail relay: hostname and port; credentials; permitted "From" address; and can it deliver to recipients outside DMRC?	Certificate dispatch goes to candidates — students on personal addresses. If the relay is internal-only, that flow cannot work at all.
13	Does any other DMRC system need to call this portal's API?	The current model assumes a person in a browser; a program would need a service account.
14	Naming convention for databases, or is <code>dmrc_internship_portal</code> acceptable?	Two lines change if not.
15	Where should application logs be written? Is there a central location?	Currently written beside the application.
16	Who owns and maintains this application after handover?	Currently undefined.

5. Security posture

- **Authorisation is decided server-side on every request**, read from the `users` table. Nothing the browser sends is trusted. All 28 endpoints are guarded; there is no unprotected endpoint. Six accept any known employee (the referral portal); the other twenty-two require a specific dashboard role.
- Issuing offer letters and certificates is restricted to HR-APP. Account management, cycle configuration and archives are restricted to SYS-ADMIN. Clearance and handover are HR-OPS and SYS-ADMIN.
- **Uploaded candidate documents** are stored outside every directory the web server serves. One authenticated endpoint reaches them: it checks the caller's role, expires the link after ten minutes, streams the file, and writes every access to the audit ledger.
- **Generated letters and certificates** are stored the same way, for the same reason.
- **Signature images** are stored more strictly again: served only to the officer they belong to and to a SYS-ADMIN.
- **Departed employees lose access automatically.** `employees.is_active` is checked during identity resolution, so a leaver is refused while every record naming them stays intact.
- **Django's built-in admin is deliberately not routed.** It would be a second, parallel username-and-password login unrelated to the employee directory. Nothing in the project uses it.
- **No runtime internet dependency.** Frontend libraries are held locally at pinned versions, verified offline.
- **Two production safety checks** refuse to start the server if the secret key is still the development default, or if the development identity provider is still configured.
- Errors and warnings are written to a rotating log file (§7).

Two items for your security team

1. **Django 4.2 reached end of extended support on 7 April 2026** and receives no further security patches. Django 5.2 LTS is supported until April 2028. I would rather complete this upgrade before go-live than after; it needs the server's Python version (Q9) to scope.
2. **The identity token travels in a URL.** URLs are recorded in browser history, in proxy and web-server access logs, and in anything a user copies or forwards. Encryption protects the contents but says nothing about freshness — so a captured URL replays indefinitely unless the token expires. Hence Q4. It would also be worth checking whether the API server's access logs record query strings.

Two things you should know rather than discover

- The data export endpoint is available to **all three dashboard roles**, including the least privileged. If exports should be restricted to administrators, say so and it is a one-line change.

- The audit ledger is an ordinary database table. It is protected by database access control, not by design — it is not tamper-proof against someone with direct database access.

6. Deployment — the steps that are easy to miss

DB/Intern_Portal.sql is the authoritative schema. It creates the database, all 27 tables, and seeds reference data.

⚠ **Run Intern_Portal.sql exactly once, against an empty database.**

It begins by dropping every table it creates. Re-running it against a database holding live data will delete that data permanently.

⚠ **Do not run anything in DB/history/ or DB/dev_only/.**

history/ holds the development migration record. Every change it makes is already in the master script, and one file uses TiDB-specific syntax that fails on MySQL outright. dev_only/ holds reset and seeding utilities — one inserts fictitious employees. Both folders carry a README explaining this.

⚠ **Run Django's migrations with --fake on a new installation.**

```
python manage.py migrate portal --fake
python manage.py migrate
```

The portal's three migrations issue raw SQL to upgrade a database built *before* those changes existed. Intern_Portal.sql already contains their result, so running them normally against a fresh database fails — migration 0002 adds columns the master script already has, and 0003 drops columns it never created. The first command records them as applied without executing them; the second then creates only Django's own internal tables. Order matters. On an existing database that predates these changes, run them normally.

Django is configured never to alter the schema — all models are declared unmanaged.

The schema also ships five analytical views (vw_hr_application_status_tracker and four others). The portal itself never reads them; they exist so that a reporting tool or Excel can be pointed at the database directly.

Then — the portal will not work until both of these are done

A. Populate employees. This portal does not own employee identity: it reads that table and never writes to it. On an empty table every request is refused with a 401, including the Phase 1 referral portal. The employee code must be *exactly* the string your login system returns.

```
INSERT INTO employees (employee_code, full_name, designation, department_id)
VALUES ('14086', 'NAME AS IN DIRECTORY', 'DESIGNATION AS IN DIRECTORY',
       (SELECT department_id FROM departments WHERE department_name = 'IT'));
```

Official email is optional. The designation is printed beneath the signature on every offer letter that person signs, so it must be kept current.

Once the directory connection in Q7 is established, this becomes an automated nightly sync rather than manual inserts.

B. Create the first SYS-ADMIN. Dashboard accounts are created from the IAM screen, which requires the caller to already hold the SYS-ADMIN role. The first account therefore cannot be created through the interface:

```
INSERT INTO users (role_id, employee_id, username, email)
VALUES ((SELECT role_id FROM roles WHERE role_name = 'SYS-ADMIN'),
       (SELECT employee_id FROM employees WHERE employee_code = '14086'),
       'chosen.username', 'person@dmrc.org');
```

On sequencing: this is part of bringing the system up, not something to do afterwards. Until that row exists, no cycle can be created, no document configured and no other account provisioned — the dashboard is inert.

Two operational notes

- **Never start this with manage.py runserver on a server.** That turns on development mode, which disables both safety checks and re-enables the identity provider that trusts a request header. Setting DEBUG=False in .env — which the example file marks as required — makes that command fail loudly instead.
- **Restrict .env** so that only the account running the application can read it. It holds the database password in plain text.

7. File storage, backup and logging

Uploaded documents, generated letters and signature images are stored on the **server filesystem**, outside the web root — not in the database:

Back End/protected_documents/	candidate uploads
Back End/generated_documents/	offer letters and certificates
Back End/signatures/	officer signature images
Back End/quarantine/	superseded and rejected files

Created automatically on first use. They require:

1. **Persistent storage** surviving redeployment.
2. **Backup together with the database.** A database-only backup restores records pointing at files that no longer exist, and closing an archive cycle fails when a referenced file is missing.
3. **Exclusion from any temporary-file cleanup policy.**

A restore has never been tested. I would suggest testing one during the pilot rather than after it.

Logging: warnings and errors are written to Back End/logs/portal.log, rotating at 5 MB with five copies retained. LOG_DIR in .env redirects this to a central location (Q15).

8. Known gaps — stated plainly

#	Gap	Status
1	61 hardcoded developer addresses (<code>http://127.0.0.1:8000</code>) — 47 in the dashboard script, 13 in the referral script, 1 in the dashboard HTML. Every one fails on the intranet.	Mine to fix, about one day including testing every screen. Blocked on Q5, Q6 and Q8 — the token arrangement determines whether each call must also carry something.
2	Automated email notifications. Certificate dispatch is currently recorded as PENDING and the notifications table is never written to.	In development. My manager approved delivering this after integration, so that DMRC IT can begin testing now rather than waiting. Blocked on Q12.
3	Django 4.2 out of security support (\$5).	Mine to fix, blocked on Q9.
4	No upload size limit on candidate documents. The database column exists and is unused. File <i>types</i> are validated per document type.	Small fix, needs a limit from HR.
5	No automated test suite. Tested by hand against the full workflow.	Would be added before this handles anything beyond a pilot.
6	No API versioning or rate limiting.	Acceptable for a single internal consumer; worth adding if Q13 is yes.
7	Ownership after handover undefined (Q16).	For discussion.

Closed since the previous version of this document

- **Database connection settings.** Previously hardcoded to a macOS certificate path with no way to change it. Now configured through `DB_SSL` and `DB_SSL_CA` in `.env`. Awaiting only the CA path (Q3).
- **Frontend libraries loaded from public CDNs.** Testing with the internet disconnected showed both portals unusable — the referral wizard did not render at all, because Alpine.js drives every screen. All libraries are now held locally at pinned versions (Alpine 3.16.2, Bootstrap 5.3.3, Bootstrap Icons 1.11.3, Flatpickr 4.6.13) and verified offline. Neither page contacts the internet.
- **No way to deactivate a departed employee.** `employees.is_active` added and honoured during identity resolution. A leaver is refused access while every record naming them survives — the database correctly refuses to delete anyone who has ever referred a candidate.

9. Personal data

Candidate records include **Aadhaar numbers, stored in full and unencrypted**, in the `students` and `archived_applications` tables. The document catalogue flags Aadhaar as requiring the candidate's consent; the consent checkbox and the number field both follow that flag, and consent is captured at upload. Every document access is written to the audit ledger.

Whether plain-text storage meets DMRC's obligations — under the DPDP Act 2023 and the DPDP Rules 2025, and under DMRC's own policy as a State instrumentality — is a decision for DMRC, not for me. **If encryption, masking, or a shorter retention period is required, tell me the requirement and I will implement it.** The same policy should be applied to the file storage directories in §7, which hold Aadhaar document images.

10. Immediate next steps

1. Answers to Q1-Q3 — the three that block work I could otherwise start now.
2. Identify who I should speak to about the token implementation.
3. Agree what "working pilot" means, so we both know when we are finished.

Prepared so that time is spent on decisions rather than description.